Fraud Detection in Credit Card Transactions

AI & ML Internship Project - Elevate Labs

1. Introduction

Credit card fraud causes large financial losses every year, and banks need to catch fraudulent transactions within seconds without blocking genuine customers. The difficulty is that fraud is extremely rare: in the dataset used here only 492 of 284,807 transactions (0.17%) are fraudulent. A model that says 'legit' every time would be 99.8% accurate and completely useless, so accuracy is the wrong yardstick and the problem needs special handling of class imbalance.

2. Abstract

This project builds and compares several fraud detectors on the Kaggle credit card fraud dataset (European cardholders, two days of transactions). Two unsupervised anomaly detectors (Isolation Forest, Local Outlier Factor) were compared with supervised models (Logistic Regression and XGBoost, the latter with class weighting and with SMOTE oversampling). Models were judged on precision, recall, F1, ROC-AUC and PR-AUC. The best model, XGBoost with class weighting, reached PR-AUC 0.88 with precision 0.87 and recall 0.84 on a held-out test set, and it was wrapped in a Streamlit web app for live prediction.

3. Tools Used

Python, Pandas, NumPy, Scikit-learn (RobustScaler, Isolation Forest, LOF, Logistic Regression, metrics), XGBoost, imbalanced-learn (SMOTE), Matplotlib, Joblib (saving the model) and Streamlit (web app).

4. Steps Involved in Building the Project

1. **Data:** loaded 284,807 transactions (Time, Amount, 28 anonymized PCA features V1-V28, Class label); no missing values.
2. **Splitting:** stratified 80/20 train/test split, plus a small validation slice of the training data used only to choose the decision threshold.
3. **Preprocessing:** scaled Time and Amount with RobustScaler (Amount ranges from 0 to about 25,000 with heavy outliers), fitting the scaler on training data only to avoid leakage.
4. **Anomaly detection:** trained Isolation Forest and Local Outlier Factor without using labels, with contamination set to the fraud rate.
5. **Supervised models:** Logistic Regression (balanced weights), XGBoost with scale_pos_weight, and XGBoost trained on SMOTE-balanced data.
6. **Threshold tuning:** picked the probability cut-off that maximizes F1 on the validation set instead of using 0.5.
7. **Evaluation:** ROC and precision-recall curves, confusion matrix and feature importance on the untouched test set (56,962 transactions, 98 frauds).
8. **Deployment:** saved the model and built a Streamlit app that scores sample transactions, manually entered values, or an uploaded CSV.

Model	Precision	Recall	F1	ROC-AUC	PR-AUC
Isolation Forest	0.295	0.337	0.314	0.959	0.218
Local Outlier Factor	0.027	0.031	0.029	0.780	0.020
Logistic Regression	0.062	0.908	0.116	0.974	0.719
XGBoost + SMOTE	0.974	0.775	0.864	0.979	0.867
XGBoost (class weight) - final	0.872	0.837	0.854	0.977	0.881

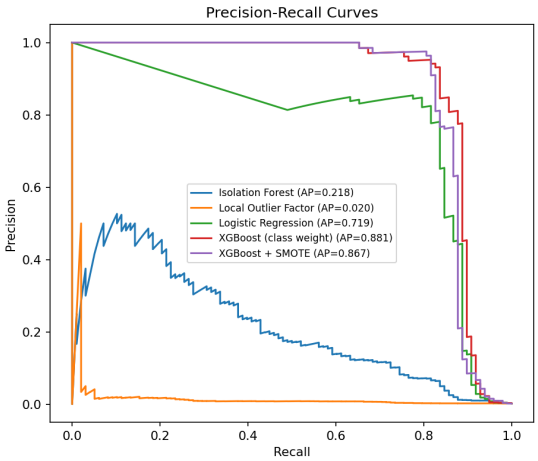


Fig. 1: Precision-recall curves of all models on the test set

5. Conclusion

Supervised XGBoost clearly beat the unsupervised detectors: on the test set it caught 82 of 98 frauds while raising only 12 false alarms among 56,864 genuine transactions. Logistic Regression found most frauds but flagged far too many legitimate transactions (precision 0.06), which shows why precision-recall metrics matter more than ROC-AUC here. Limitations: the features are anonymized so results cannot be explained in business terms, the split is random rather than time-based, and with only 98 test frauds the scores are somewhat noisy. Future work includes time-based validation, cost-sensitive thresholds and model monitoring for drift.